

Aufgabenblatt 6

Graphenalgorithmen

Wichtige Ankündigungen

- Es gibt drei Aufgabentypen:
 - **Tutorium** → Besprechung im Tutorium
 - **Hausaufgabe** → Eigenarbeit mit Hilfe in Rechnerübungen
 - **Klausurvorbereitung** → Optionale Klausurübung

Abgabe (bis 23.06.2025 23:59 Uhr)

Die folgenden Dateien werden für die Bepunktung berücksichtigt:

Geforderte Dateien:

Blatt06/src/Maze.java	Aufgabe 3.3, 3.5 & 3.6
Blatt06/src/DepthFirstPaths.java	Aufgabe 3.1 & 3.2
Blatt06/src/RandomDepthFirstPaths.java	Aufgabe 3.4

Als Abgabe wird jeweils nur die letzte Version im main branch in git gewertet.

Aufgabe 1: Graphenalgorithmen (Tutorium)

Betrachten Sie diesen U-Bahnplan:



- 1.1 Auf welche Art und Weisen können Sie diesen U-Bahnplan als Graph darstellen. Wie könnte man den Graphen in einem Inputfile darstellen?

Lösung

Darstellung als Graph:

- Stationen als Knoten, Strecken zwischen den Stationen als Kanten

mögliche Inputfiles:

- Adjazenzmatrix
- Knotenpaare, die zu Kanten werden
- Listen: Knoten mit allen benachbarten Kanten

- 1.2 Wie könnte man die Klasse Graph implementieren? Betrachten Sie unterschiedliche Lösungen und machen Sie sich deren Vor- und Nachteile klar.

Lösung

1. Nodes und Edges als einzelne Klassen (ähnlich dem Aufbau der 1. HA) - Struktur ist über die Edges gespeichert
2. Adjazenzmatrix im Graphen
3. Adjazenzlisten für jeden Knoten
4. Knoten werden als Doppelt verkettete Pfeillisten (DCAL) gespeichert

Vor- und Nachteile:

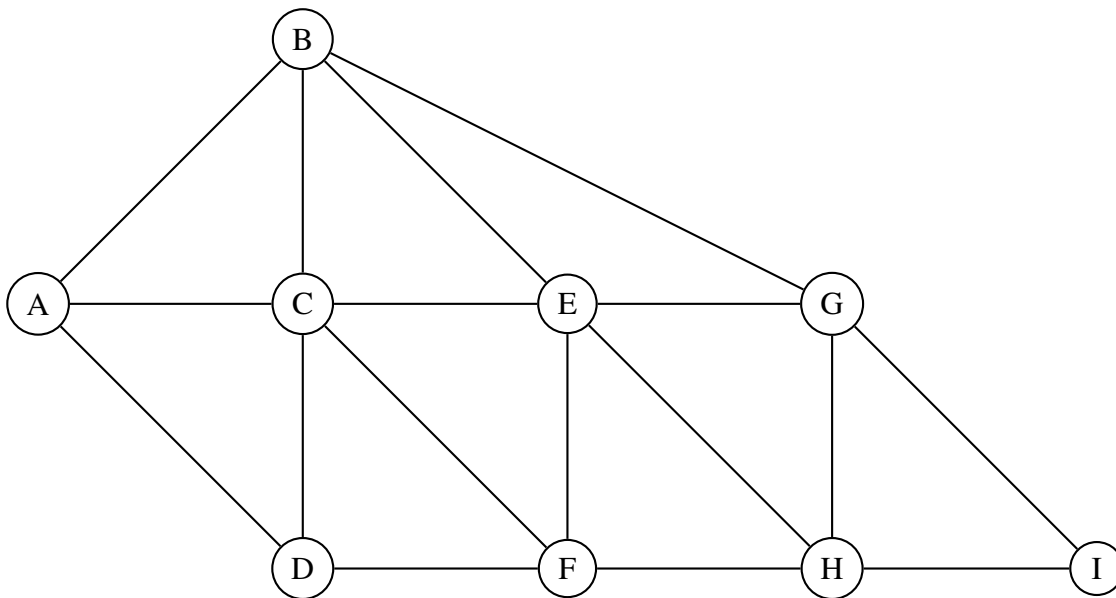
1. anschauliche Implementierung
2. Speicherbedarf $\Theta(V^2)$, unabhängig von der Anzahl der Kanten. Laufzeit beträgt $\mathcal{O}(1)$ zum Prüfen, ob v und w adjazent sind; $\mathcal{O}(V)$ zum Iterieren über die Nachbarknoten von v . Daher geeignet für dichte, statische Graphen.
3. Speicherbedarf von Adjazenzlisten ist in $\Theta(V + E)$. Laufzeit $\mathcal{O}(\text{grad}(v))$ zum Prüfen, ob v und w adjazent sind; $\mathcal{O}(\text{grad}(v))$ zum Iterieren über die Nachbarknoten von v . Daher geeignet für dünne, statische Graphen.
4. Flexibel, da Knoten hinzugefügt und entfernt werden können: geeignet für dünne dynamische Graphen

1.3 Wie funktionieren die Breiten- und die Tiefensuche? Worin bestehen die Unterschiede?

Lösung

Breitensuche geht zuerst in die Breite; Tiefensuche zuerst in die Tiefe. Siehe auch Aufgabe 2.

Aufgabe 2: BFS und DFS (Tutorium)



2.1 Führen Sie die Breitensuche auf dem gegebenen Graphen aus. Fangen Sie bei Knoten A an und notieren Sie alle Knoten in der Reihenfolge, in der sie von der Breitensuche in die Warteschlange geschrieben werden.

Gehen Sie dabei davon aus, dass in jedem Knoten die benachbarten Knoten in alphabetischer Reihenfolge abgearbeitet werden. Zum Beispiel wird die Kante $F-C$ vom Algorithmus vor der Kante $F-D$ bearbeitet.

Lösung

Knoten:

A, B, C, D, E, G, F, H, I

Entwicklung der Queue:

[]

[A]

[B, C, D]

[C, D, E, G]

[D, E, G, F]

[E, G, F]

[G, F, H]

[F, H, I]

[H, I]

[I]

[]

- 2.2** Führen Sie die Tiefensuche auf dem gegebenen Graphen aus. Fangen Sie bei Knoten A an und notieren Sie alle Knoten in der Reihenfolge, in der sie von der Tiefensuche entdeckt werden. Gehen Sie dabei davon aus, dass in jedem Knoten die benachbarten Knoten in alphabetischer Reihenfolge abgearbeitet werden. Zum Beispiel wird die Kante *F-C* vom Algorithmus vor der Kante *F-D* bearbeitet.

Lösung

Preorder:

A, B, C, D, F, E, G, H, I

Postorder:

I, H, G, E, F, D, C, B, A

Entwicklung des Stack:

[]

[A]

[A, B]

[A, B, C]

[A, B, C, D]

[A, B, C, D, F]

[A, B, C, D, F, E]

[A, B, C, D, F, E, G]

[A, B, C, D, F, E, G, H]

[A, B, C, D, F, E, G, H, I]

[A, B, C, D, F, E, G, H]

[A, B, C, D, F, E, G]

[A, B, C, D, F, E]

[A, B, C, D, F]

[A, B, C, D]

[A, B, C]

[A, B]

[A]

[]

Knoten werden erst aus dem Stack entfernt, wenn Sie fertig abgearbeitet sind. Gäbe es noch einen Knoten J mit der Kante **B-J**, würde dieser erst am Ende beim Abarbeiten des Knotens **B** gefunden werden.

Aufgabe 3: Für die Hausaufgaben wird keine Lösung bereitgestellt.

Was Sie nach diesem Blatt wissen sollten:

- Was (ungerichtete und gerichtete) Graphen sind und welche Repräsentationen von Graphen es gibt.
- Was der Zweck einer Breiten- bzw. einer Tiefensuche ist.
- Wie die Breiten- und die Tiefensuche auf Graphen funktionieren
- Was die Breiten- und die Tiefensuche unterscheidet
- Wie man mit der Breiten- bzw. Tiefensuche einen Zyklus im Graphen finden kann und wie man Zusammenhangskomponenten erkennen kann
- Was die Haupt- und die Nebenreihenfolge sind
- Was topologische Sortierung ist